Supplementary Material for "Simple Trees Beat Deep Networks at Attributing VMProtect-Protected Malware: A Leakage-Free, Forgery-Aware Evaluation"

Xubo Fang*, Haochen Dong, Lei Zhong, and Jiaqing Wang

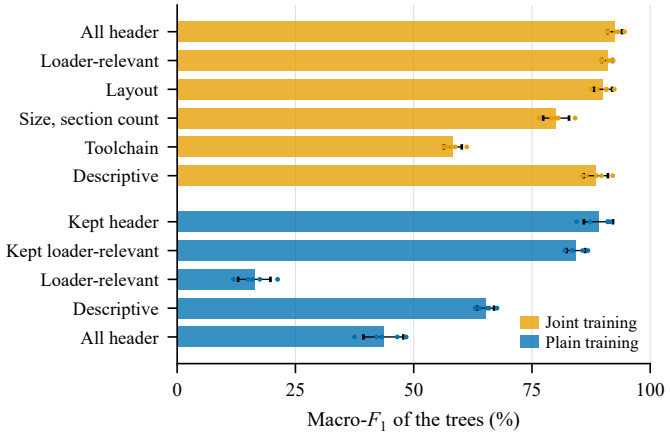


Fig. S1. Protected-test macro-F_1 of gradient-boosted trees on subsets of the header fields on folds grouped by build (bars: means over five folds; caps: standard deviation; dots: folds; seed 0). Joint training uses plain and protected executables; plain training uses plain executables only. Kept fields are those that VMProtect kept in at least 95% of the training pairs.

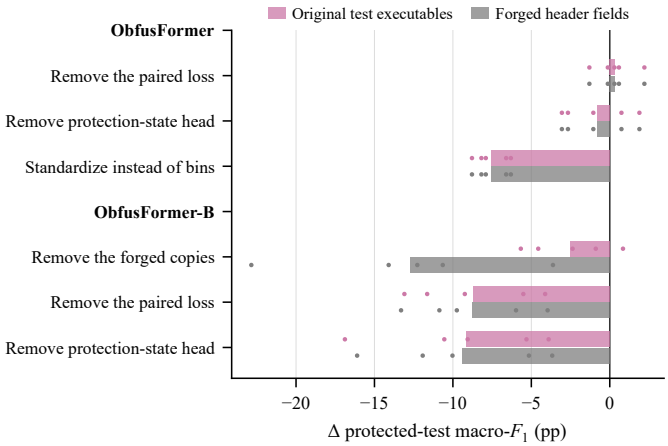


Fig. S2. Ablation of ObfusFormer. Change in the protected-test macro-F_1 (percentage points) when one part of the training is removed, relative to the configuration it is removed from, on the original test executables and on executables with forged descriptive header fields. ObfusFormer uses the structure view and ObfusFormer-B the byte views. The bars are means over the five folds and the dots the folds (Joint protocol, folds grouped by build, seed 0).

This document supplements the main paper. Section SI analyzes which header fields carry the family signal, Section SII ablates the training of ObfusFormer, the strongest neural baseline, and Section SIII reports a neural architecture search over the ObfusFormer design space. Section, equation, and reference numbers of the main paper are used as written there.

SI. Attribution on Header-Field Subsets

Fig. S1 reports the protected-test macro-F_1 of the gradient-boosted trees on subsets of the header fields, on the folds grouped by build (Section V-B of the main paper). The layout of a program carried most of the signal: the trees on the layout fields alone reached 90.0%, against 58.3% on the toolchain fields (the machine, the alignments, the image base, and the stack and heap sizes) and 80.1% on the coarse file sizes and section count alone. Trained on plain executables only, the trees on all loader-relevant fields reached 16.3% on protected ones; restricted to the 49–52 fields that VMProtect kept in at least 95% of the training pairs, they recovered to 89.1%.

The authors are with [Affiliation], [City], [Country].
*Corresponding author: Xubo Fang (e-mail: [e-mail]).

SII. Ablation of ObfusFormer

Fig. S2 shows the change in the protected-test macro-F_1 of ObfusFormer when one part of its training is removed, relative to the configuration it is removed from (Section V-D of the main paper). Removing the paired loss or the protection-state head changed the protected-test macro-F_1 of ObfusFormer by +0.2 and −0.5 points, less than the seed band, whereas standardization instead of the bin encoding changed it by −7.3 points. For ObfusFormer-B, the forged copies helped only on the forged samples (−9.6 points).

SIII. Neural Architecture Search over ObfusFormer

To check that the comparison does not depend on a single hand-chosen ObfusFormer configuration, we ran a neural architecture search over its design space, varying the depth, the width, the number of attention heads, the set of views, and the tokenizers. Each configuration was trained on the training split and selected on a held-out validation split; the protected test set was never used during the search. The configuration with the highest validation score was then retrained and evaluated once on the protected test set under the Joint protocol.